

İŞLETİM SİSTEMİ PROJE ÖDEVİ RAPOR

Sayfalama Nedir?

İşletim sistemlerinde sayfalama RAM ve programların bellek alanlarını eşit boyutlu küçük parçalara (sayfa ve çerçeve) bölerek hafıza kullanımını optimize eden ve sanal belleği mümkün kılan temel bir bellek yönetim tekniğidir.

Sayfalama Nasıl Çalışır?

- Sayfa : Çalışan programların mantıksal bellek adres uzayında böldüğü sabit boyutlu veri bloklarıdır.
- Çerçeve : Ana belleğin sayfa boyutlarıyla birebir aynı olacak şekilde bölündüğü fiziksel bloklardır.
- Sayfa Tablosu : İşletim sisteminin her bir sürecin sayfalarını hangi RAM çerçevelerine denk geldiğini eşleştirdiği ve takip ettiği bir haritadır.

Sayfalamanın Temel Özellikleri ve Avantajları

- Sürekli Bellek Zorunluluğunun Kalkması: Bir programın bellekte kapladığı alanın ardışık olma zorunluluğu ortadan kalkar. Programın bölümleri RAM'deki dağınık, boş çerçevelere esnek bir şekilde dağıtılabilir.
- Sanal Bellek Desteği : RAM dolduğunda, acil ihtiyaç duyulmayan sayfalar ikincil depolama birimine taşınır.
- Verimlilik: İşletim sistemleri bellek parçalanmasını en aza indirir.

Page Fault ve Page Hit Nedir?

Sayfa hatası (page fault) sanal bellek tekniğinde başvuru alan veri o anda ana bellekte bulunamıyorsa oluşur ve aranan sayfa ana bellekte mevcut değil demektir.

Bellekte tutulan sayfa tablosu sanal bellek adresinin numarasına göre dizinlenmiştir ve ona karşılık gelen gerçek sayfa numarasını içerir. Her program sanal adres uzayını ana bellekteki bellek uzayına dönüştüren kendine ait bir sayfa tablosuna sahiptir. Sayfa tablosu ana bellekte mevcut olmayan sayfaların kayıtlarını da tutabilir. Her sayfa tablosunda geçerli bit (1 veya 0) tutulur. Eğer bu bit mantıksal sıfıra eşit ise sayfa ana bellekte mevcut değil demektir ve “sayfa hatası (page fault)” oluşur.

Page hit (sayfa isabeti), çalışan bir programın ihtiyaç duyduğu veri veya talimatları aradığında, bunların fiziksel bellekte (RAM) zaten bulunması durumuna verilen isimdir. Bu süreç sistemin hızlı çalışmasını sağlar.

Algoritmaların Çalışma Mantığı

FIFO, yönetimi en kolay ve maliyeti en düşük algoritmadır. Bellek yönetim birimi RAM'deki çerçeveleri takip etmek için arka planda bir kuyruk veri yapısı tutar. Belleğe yüklenen her yeni sayfa bu kuyruğun sonuna eklenir. Eğer RAM tamamen dolarsa ve yeni bir sayfa isteği gelirse kuyruğun en başındaki yani bellekte en uzun süredir ikamet eden sayfa kapının önüne koyulur.

LRU, yakın geçmişe bakarak yakın geleceği tahmin etme ilkesine dayanır. FIFO gibi sayfaların belleğe sadece giriş anıyla ilgilenmez asıl odaklandığı şey sayfaların son kullanım anıdır. Algoritma bellek dolduğunda geçmişe doğru bir tarama yapar ve kronolojik olarak en uzun süredir işlemci tarafından yüzüne bakılmayan, erişilmeyen sayfayı kurban seçerek bellekten atar.

Optimal algoritma doğrudan geleceğe odaklanır. Temel kuralı şudur: Bellek dolduğunda ve bir sayfa atılması gerektiğinde, RAM'deki sayfalar incelenir ve gelecekte en geç kullanılacak olan veya gelecekte bir daha asla kullanılmayacak olan sayfa tespit edilerek bellekten atılır.

LFU'da sistem her sayfanın bellekte kalma süresi boyunca ne kadar aktif kullanıldığını izler. Bellek dolduğunda geçmişe döner ve o ana kadar en az hit alan, yani referans edilme sıklığı en düşük olan sayfayı gözden çıkararak bellekten atar.

Deneysel Sonuçlar

Buradaki referans ve çerçeve sayısını hocamızın derste anlattığı pdf'ten örnek olarak aldım.

Deney 1:

```
Referans dizisi girin (Bitirmek için -1): 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 -1
```

çerçeve sayısı:3

Algoritma	Page Fault	Page Hit	Fault Oranı	Hit Oranı
FIFO	15	5	75.00	25.00
LRU	12	8	60.00	40.00
Optimal	9	11	45.00	55.00
LFU	11	9	55.00	45.00

çerçeve sayısı:4

Algoritma	Page Fault	Page Hit	Fault Orani	Hit Orani
FIFO	10	10	50.00	50.00
LRU	8	12	40.00	60.00
Optimal	8	12	40.00	60.00
LFU	9	11	45.00	55.00

çerçeve sayısı:5

Algoritma	Page Fault	Page Hit	Fault Orani	Hit Orani
FIFO	9	11	45.00	55.00
LRU	7	13	35.00	65.00
Optimal	7	13	35.00	65.00
LFU	7	13	35.00	65.00

3 Çerçeve :

- Bu aşamada algoritmalar arasındaki farklar çok belirgin.
- Optimal (9) ile FIFO (15) arasında ciddi bir uçurum var. Bellek daraldıkça, Page Fault çok daha yüksek oluyor.
- LRU, kısıtlı bellekte FIFO'ya göre daha iyi performans sergileyerek yerellik ilkesinin önemini gösteriyor.

4 Çerçeve :

- Hata oranlarını inanılmaz bir hızla düşürmüştü. Örneğin FIFO'da hata sayısı 15'ten 10'a inmişti.
- 4 çerçeve kullanımında LRU, Optimal algoritmayı yakalamış (ikisi de 8 hata). Bu, referans dizisinin o anki yapısının LRU mantığına tam uyum sağladığını gösterir.

5 Çerçeve :

- Bellek 5 çerçeveye çıktığında, algoritmaların çoğu (LRU, Optimal, LFU) 7 hata seviyesinde birbirine yaklaşıyor.
- Belleği belirli bir noktadan sonra artırmak performansı artık aynı hızda iyileştirmez. Çünkü bazı sayfa hataları zorunludur ve bellek ne kadar büyük olursa olsun bu hatalar engellenemez.

Hata sayısı sürekli azaldığı için sistemin sağlıklı bir bellek yönetimi sergilediğini söyleyebiliriz.

Deney 2:

Referans dizisi girin (Bitirmek için -1): 7 0 1 2 0 3 0 4 2 3 0 3 2 -1
Cerceve sayısı: 3

1 - FIFO
2 - LRU
3 - Optimal
4 - LFU
5 - Tumunu Calistir ve Karsilastir
0 - Cikis
Secim: 5

Algoritma	Page Fault	Page Hit	Fault Orani	Hit Orani
FIFO	10	3	76.92	23.08
LRU	9	4	69.23	30.77
Optimal	7	6	53.85	46.15
LFU	8	5	61.54	38.46

Referans dizisi girin (Bitirmek için -1): 1 2 3 4 1 2 5 1 2 3 4 5 -1
Cerceve sayısı: 3

1 - FIFO
2 - LRU
3 - Optimal
4 - LFU
5 - Tumunu Calistir ve Karsilastir
0 - Cikis
Secim: 5

Algoritma	Page Fault	Page Hit	Fault Orani	Hit Orani
FIFO	9	3	75.00	25.00
LRU	10	2	83.33	16.67
Optimal	7	5	58.33	41.67
LFU	10	2	83.33	16.67

İki farklı referans dizisini karşılaştırdığımızda, algoritmaların başarısının tamamen verinin yapısına bağlı olduğu görülüyor. İlk senaryoda sayfalar sık tekrarlandığı için yerelliği izleyen LRU ve LFU algoritmaları FIFO'yu geride bırakarak daha verimli çalışmıştır. Ancak ikinci senaryodaki döngüsel istek yapısı, LRU'nun yanılmasına ve şaşırtıcı şekilde FIFO'dan daha kötü sonuç vermesine neden olmuştur. Her iki durumda da Optimal algoritma ulaşılacak en düşük hata sayısına ulaşarak teorik üstünlüğünü korumuştur. Sonuç olarak bellek miktarı sabit olsa bile erişim sırasındaki değişimler sistem performansını doğrudan etkilemektedir. Bu veriler tek bir en iyi algoritma olmadığını iş yüküne göre tercihin değişmesi gerektiğini kanıtlar.

Yorum ve Karşılaştırma

Referans dizisi girin (Bitirmek için -1): 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
-1

Algoritma	Page Fault	Page Hit	Fault Oranı	Hit Oranı
FIFO	15	5	75.00	25.00
LRU	12	8	60.00	40.00
Optimal	9	11	45.00	55.00
LFU	11	9	55.00	45.00

Tablodaki veriler, kısıtlı çerçeve sayısının tüm algoritmalarda yüksek hata oranlarına yol açtığını göstermektedir. **FIFO** sayfa özelliklerini dikkate almadığı için en verimsiz performansı sergilemiştir. **Optimal** algoritma geleceği öngörebilme avantajıyla en düşük hata sayısına ulaşarak sistemin verimlilik sınırını belirlemiştir. **LRU ve LFU** ise geçmiş verileri kullanarak FIFO'ya göre daha yüksek isabet (**Page Hit**) oranları yakalamıştır. Yüksek hata oranları mevcut bellek miktarının referans dizisini karşılamakta zorlandığını ve yoğun disk erişimi gerektiğini kanıtlar. Genel olarak kullanım geçmişini analiz eden algoritmalar basit sıralama mantığına göre çok daha başarılı sonuçlar vermiştir.

- En düşük page fault hangi algoritmada elde edilmiştir?

En düşük hata sayısı **Optimal** algoritmada elde edilmiştir çünkü bu yöntem gelecekte hangi sayfanın kullanılacağını önceden bilerek en doğru tahliye yapar.

- FIFO algoritması her zaman iyi sonuç verir mi?

Her zaman iyi sonuç vermez özellikle sayfa kullanım sıklığını ve zamanını dikkate almadığı için genellikle en yüksek hata oranına sahiptir

- LRU algoritması FIFO'ya göre hangi durumlarda avantaj sağlar?

Yakın zamanda kullanılan sayfaların tekrar kullanılma ihtimalinin yüksek olduğu durumlarda FIFO'ya göre büyük avantaj sağlar.

- Optimal algoritma gerçek sistemlerde neden doğrudan uygulanamaz?

Gerçek sistemlerde bir programın gelecekte hangi sayfaları isteyeceği önceden kesin olarak bilinemediği için Optimal algoritma doğrudan uygulanamaz ancak diğer algoritmaları kıyaslamak için bir referans noktası olarak kullanılır.

- Çerçeve sayısı artırıldığında page fault sayısı nasıl değişmiştir?

Çerçeve sayısı artırıldığında, fiziksel bellekte daha fazla sayfa tutulabildiği için Page Fault sayısı azalmış ve sistemin isabet oranı (Page Hit) artmıştır.